

Improving Solutions Quality for Large-Scale Datasets of the Traveling Salesman Problem

Trần Thành Đạt

VNU University of Engineering and Technology

ttdat.vnu@gmail.com

Abstract

Bài toán người chào hàng (Travelling Salesman Problem - TSP) là một bài toán tối ưu tổ hợp kinh điển, với nhiều ứng dụng thực tiễn [1], [2], [3]. Trong những bộ dữ liệu lớn (trên 30,000 đỉnh), TSP chủ yếu được giải bằng các thuật toán heuristic, và các thuật toán metaheuristic mô phỏng tự nhiên. Trong những công bố gần đây của thuật toán tối ưu đàn kiến (Ant Colony Optimization - ACO) cho bài toán TSP với bộ dữ liệu lớn có nhiều triển vọng [4], [5]. Trong nghiên cứu này, tác giả tổng hợp và phân tích kỹ lưỡng các tính chất và cải tiến từ trước tới nay của ACO, phân tích và đề xuất thêm 2 cải tiến mới: kiểm soát hành vi của kiến qua ép thay đổi đường đi và thay đổi lượng kiến cơ động theo thời gian khi kết hợp với cập nhật Max-Min tròn [6]. Thực nghiệm cho thấy thuật toán được đề xuất đạt được sai số nhỏ hơn 0.3% trên bộ dữ liệu Art TSP với số đỉnh từ 100,000 đến 200,000 đỉnh [7], và dưới 0.8% cho những bộ dữ liệu khác từ 33,810 đến 100,000 đỉnh. Thực nghiệm cũng cho thấy thuật toán mới tốt hơn so với các thuật toán ACO tốt nhất hiện nay ([8], [9], [10], [4], [5]), đặc biệt, thuật toán đã tìm được lời giải tốt nhất mới cho một số bộ dữ liệu.

1. Giới thiệu

1.1. Bài toán người chào hàng

Bài toán người chào hàng (Travelling Salesman Problem - TSP) là một bài toán tối ưu tổ hợp kinh điển, đồng thời cũng là một trong những bài toán tối ưu tổ hợp được nghiên cứu nhiều nhất từ trước đến nay [1]. Bài toán xét một tập các thành phố và khoảng cách giữa chúng, một người bắt đầu từ một thành phố bất kỳ và thăm lần lượt các thành phố, mỗi thành phố đúng một lần, trước khi quay trở về thành phố ban đầu. Đường đi tạo thành một chu trình Hamilton (Hamiltonian Cycle), và mục tiêu của bài toán là tìm ra chu trình Hamilton tối ưu (Optimal Hamilton Cycle) với tổng độ dài nhỏ nhất. TSP có thể được ứng dụng để giải quyết rất nhiều bài toán

trong đời sống, chủ yếu là trong kĩ thuật và logistic, các bài toán tiêu biểu như VRP (bài toán tối ưu hóa vận tải) [11], bài toán phủ đỉnh [12], nhận biết cạnh trong ảnh [13], hay thiết kế mạch điện tử [14] tất cả đều có thể đưa về TSP để giải quyết.

Bài toán TSP là một bài toán thuộc lớp NP-Khó, không có thuật toán đơn định với độ phức tạp đa thức giải bài toán [1]. Do đó, TSP thường được giải bằng những thuật toán heuristic cho ra kết quả gần tối ưu. Trong đó, những thuật toán meta-heuristic mô phỏng tự nhiên, tiêu biểu là thuật toán tối ưu đàn kiến (Ant Colony Optimization [15]) đã được sử dụng để tiếp cận TSP, đem lại kết quả tốt.

2. Tối ưu đàn kiến cho bài toán người chào hàng

2.1. Thuật toán kinh điển

Tối ưu đàn kiến (ACO) thể hiện cho không phải một thuật toán cụ thể, mà là một nhóm những thuật toán mà đặc điểm chính là mô phỏng lại cách tìm đường đi từ tổ đến nguồn thức ăn của kiến trong môi trường tự nhiên [15]. Một số thuật toán ACO kinh điển có thể kể đến như Ant System (Dorigo et al. 1996 [16]), Ant Colony System (Dorigo et al. 1997 [17]), Max-Min Ant System (Dorigo et al. 1997 [18]).

Bài toán người chào hàng là một trong những bài toán tối ưu tổ hợp đầu tiên được áp dụng giải bằng ACO [16], và cho đến nay vẫn là tiêu chuẩn để thực nghiệm đánh giá những thuật toán ACO mới [19]. Trong thuật toán ACO, đàn kiến nhân tạo di chuyển quanh đồ thị $G = (V, E)$, đi từ đỉnh hiện tại và chọn để đi đến một đỉnh chưa thăm tiếp theo, dựa theo thông tin về vết mùi. Với mọi cạnh $(i, j) \in E$, lưu lại một giá trị $\tau_{ij}(t)$ định nghĩa vết mùi được lưu lại ở cạnh (i, j) tại thời điểm t . Lưu lại vết mùi cho phép mô phỏng lại hành vi của kiến trong tự nhiên: kiến khi đi tìm thức ăn sẽ để lại vết mùi trên đường đi của mình, và độ dày đặc của mùi được để lại sẽ được quyết định dựa trên chất lượng của thức ăn ở đích đến. Trong ACO gốc, vết mùi được lưu lại thành một ma trận mùi với không gian $O(n^2)$.

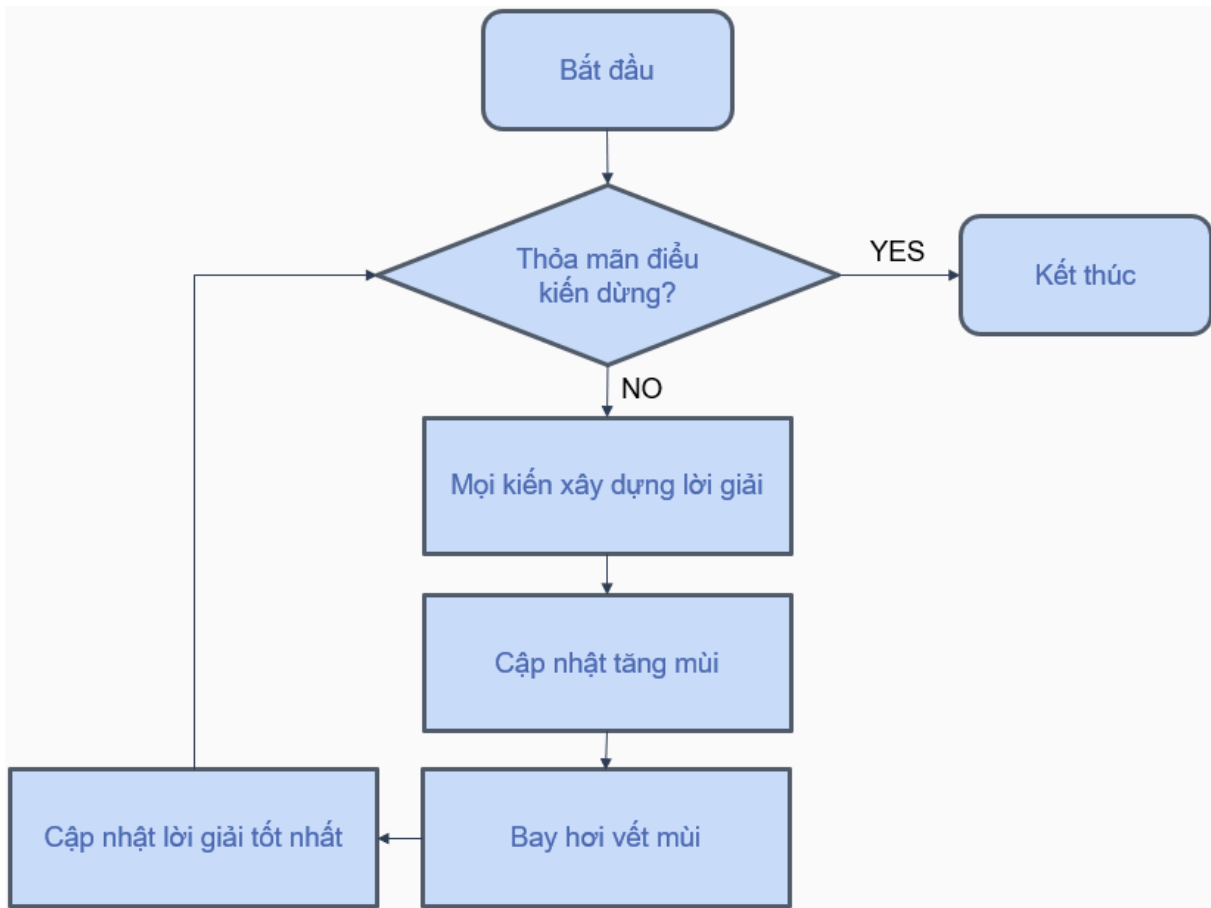
Tại bước thứ t , kiến k tại đỉnh i sẽ chọn đỉnh j với xác suất được định nghĩa như sau:

$$p_{ij}^k = \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}^\beta]}{\sum_{i \in N_k} [\tau_{ij}(t)]^\alpha [\eta_{ij}^\beta]}$$

Với N_k là tập các đỉnh chưa thăm của kiến k , $j \in N_k$, η_{ij} là thông tin heuristic của cạnh (i, j) ; α và β thể hiện sự ảnh hưởng của vết mùi và thông tin heuristic. Đối với bài toán TSP, thông tin heuristic thường là nghịch đảo của độ dài cạnh, hay $\eta_{i,j} = \frac{1}{d_{ij}}$, khiến cạnh ngắn hơn dễ có khả năng được lựa chọn hơn.

Trong bài toán TSP, thông tin heuristic không đổi trong suốt quá trình thuật toán chạy, còn giá trị vết mùi thì thay đổi liên tục dựa vào 2 bước chính trong ACO: *bay hơi vết mùi* và *cập nhật tăng mùi*. *Bay hơi vết mùi* khiến giá trị vết mùi trên mọi cạnh đều giảm, theo công thức $\tau_{ij}(t) = \tau_{ij}(t-1) * (1-p)$, trong đó p được gọi là hệ số bay hơi. Với *cập nhật tăng mùi*, hệ kiến AS gốc tăng vết mùi của mọi cạnh có kiến đi qua, với công thức $\tau_{ij}(t) = \tau_{ij}(t-1) + \delta_k$, trong đó δ_k được tính là nghịch đảo của độ dài đường đi kiến k , hay $\delta_k = \frac{1}{d_k}$. Ngoài ra, Dorigo còn đề xuất Max-Min Ant System hay MMAS [18], bổ sung thêm 2 cải tiến vào hệ kiến gốc:

1. Thuật toán đề xuất kẹp giá trị τ_{ij} bằng 2 giá trị $\tau_{\min}$ và $\tau_{\max}$, hay $\tau_{ij} = \min(\tau_{\max}, \max(\tau_{\min}, \tau_{ij}))$. Chặn trên vết mùi bằng $\tau_{\max}$ giúp thuật toán ít bị kẹt vào tối ưu cục bộ do cộng dồn vết mùi lên quá lớn, và chặn dưới vết mùi bằng $\tau_{\min}$ giúp cho mọi cạnh đều có khả năng được chọn, tăng tính khám phá của thuật toán đồng thời cũng giúp thuật toán thoát khỏi tối ưu cục bộ. Ban đầu vết mùi của mọi cạnh sẽ đều được đặt là $\tau_{\max}$.
2. Tính ưu tú của kiến: trong đó, chỉ kiến với lời giải tốt nhất trong một vòng lặp được cho phép tăng vết mùi. Cải tiến này giúp thuật toán hội tụ về lời giải tối ưu nhanh hơn, không bị loãng do tăng mùi quá lộn xộn như trước.



Lược đồ cài đặt Tối ưu đàn kiến

2.2. Hạn chế của tối ưu đàn kiến cho bài toán người chào hàng

Thuật toán tối ưu đàn kiến thuộc nhóm những giải thuật metaheuristic mô phỏng tự nhiên được áp dụng để giải quyết bài toán tối ưu tổ hợp khó, tiêu biểu là bài toán người chào hàng. Tuy nhiên, dù đạt được những kết quả tốt khi mới được đề xuất, những năm gần đây ACO thể hiện tính hạn chế khi áp dụng vào những bộ dữ liệu lớn và rất lớn. Lý do là bởi hạn chế về mặt không gian (bộ nhớ) và thời gian (độ phức tạp) của thuật toán. Thuật toán AS thuần túy được đề xuất bởi Dorigo yêu cầu lưu lại ma trận mùi với độ phức tạp không gian $O(n^2)$, độ phức tạp thời gian là $O(I * m * n^2)$, với I là số vòng lặp, m là số kiến và n là số đỉnh của bộ dữ liệu. hạn chế cài đặt và thử nghiệm ACO trên các bộ dữ liệu rất lớn. Theo Ismkhan et al. [20], cho đến năm 2017, hầu hết các công trình nghiên cứu sử dụng ACO có cụm từ “large-scale” ở tiêu đề chỉ thử nghiệm thuật toán ACO đó với bộ dữ liệu vài trăm đỉnh, cao nhất là 3000 đỉnh.

Cho đến nay, rất nhiều biến thể và tối ưu của ACO đã được nghiên cứu và phát triển, bao gồm nhưng không giới hạn Population-Based ACO (Guntsch et al. 2002 [21]), Beam ACO (Christian et al. 2010 [22]), Effective Strategies ACO (Ismkhan et al. 2017 [20]), PartialACO (Chitty et al. 2018 [10]), ACO-RPMM (Peake et al. 2019 [9]), Focused ACO (Skinderowicz et al. 2023 [4]), AddACO (Scianna et al. 2024 [23]), .. Tuy nhiên, số lượng thuật toán ACO được đề xuất và thực nghiệm trên bộ dữ liệu lớn (trên 100,000 đỉnh) vẫn còn ít và chỉ xuất hiện trong vài năm gần đây. Một số thuật toán nổi bật có thể kể đến như ACO-RPMM, ACOTSP-MF, P-ACO, PartialACO, FocusedACO.

3. Công trình liên quan

Từ đó đến nay, nhiều công trình nghiên cứu đã đề xuất một số thay đổi và tối ưu để vượt qua những giới hạn này: ACOTSP-MF [8] đề xuất thêm tập đỉnh dự phòng để tối ưu thời gian chọn đỉnh tiếp theo; P-ACO [21] hoàn toàn loại bỏ ma trận mùi, thay vào đó duy trì một tập những đường đi tốt để tái tạo lại vết mùi khi cần; ESACO [20] lưu lại ma trận mùi dưới dạng ma trận thưa, loại bỏ những cạnh có vết mùi nhỏ; cùng với nhiều nghiên cứu về hướng song song hóa ACO; chạy ACO trên GPU, .. Các công trình nghiên cứu trên đạt được một số thành tựu nhất định, nhưng hiệu suất của ACO trên những bộ dữ liệu lớn này vẫn chưa thể so sánh được với GA-EAX và LKH [8], [5], [24]. Trong những công trình trên, PartialACO [10] và FocusedACO [4] đưa ra ý tưởng đáng chú ý khi nhận biết được đường kiến đi không có nhiều thay đổi so với đường đi tốt trước đó, qua đó thông qua giới hạn lại số thay đổi kiến tạo có thể trực tiếp giảm đáng kể thời gian chạy của thuật toán, đồng thời cho phép kiểm soát tốt hơn hành vi của kiến, qua đó cải thiện đáng kể chất lượng lời giải. FocusedACO [4], [5], theo đến thời điểm hiện tại, là thuật toán ACO đưa đến kết quả tốt nhất trên bộ dữ liệu lớn trong các thuật toán ACO được thực nghiệm, với khả năng đạt được sai số với lời giải tốt nhất từ 0.4% đến 0.7% với các bộ dữ liệu từ 100,000 đến 200,000 đỉnh trong Art TSP, với thời gian chạy dưới 1h [5].

3.1. Tối ưu bộ nhớ ma trận mùi

Cản trở rõ ràng nhất để áp dụng ACO với những bộ dữ liệu lớn hơn là ma trận mùi của thuật toán, một ma trận có kích thước $n * n$. Với bộ dữ liệu $n = 10^5$ biến 32-bit, ma trận này sẽ tốn 37 GB bộ nhớ. Cho dù có đủ RAM để lưu trữ, đọc và ghi

bộ nhớ lớn như vậy sẽ ảnh hưởng đáng kể đến thời gian chạy của thuật toán. Dorigo trong bài báo của ông về ACO cho TSP [25], đề xuất sử dụng danh sách ứng cử viên cho mỗi đỉnh, qua đó mỗi đỉnh chỉ quan tâm đến một số lượng hằng số các đỉnh khác. Tiêu chí đánh giá ứng cử viên đơn giản nhất là Nearest Neighbor, tức là đỉnh kề gần nhất: trong bộ dữ liệu pcb422.tsp, đường đi tối ưu chỉ sử dụng đồ thị con chứa 6 đỉnh kề gần nhất của mỗi đỉnh, hay trong bộ dữ liệu pr2392.tsp, chỉ sử dụng 8 đỉnh [25].

3.2. Tối ưu thời gian chọn đỉnh tiếp theo

Danh sách ứng cử viên đã giảm mạnh độ phức tạp thời gian của việc chọn đỉnh tiếp theo. Vấn đề xảy ra khi tất cả những đỉnh ứng cử viên đều đã được thăm, điều đó xảy ra rất thường xuyên ở giai đoạn cuối của quá trình tạo đường đi. Lúc đó, thuật toán buộc phải duyệt tất cả những đỉnh còn lại và lựa chọn một trong những đỉnh đó dựa vào thông tin heuristic (vì những đỉnh đó không lưu thông tin mùi). Khi đó, thời gian chọn đỉnh tiếp theo lại mất $O(n)$. [8] đề xuất ACOTSP-MF, trong đó bổ sung thêm một tập những đỉnh dự phòng, qua đó khi danh sách ứng cử viên không chọn được, kiến sẽ duyệt tìm một đỉnh gần nhất trong danh sách dự phòng trước, và nếu vẫn không tìm được đỉnh chưa thăm thì mới phải duyệt n đỉnh sau. Theo thực nghiệm, tối ưu đơn giản nhưng hiệu quả này trực tiếp giảm chi phí chọn đỉnh tiếp theo đi đáng kể.

3.3. Tối ưu số lần chọn đỉnh tiếp theo (kiểm soát số thay đổi)

Nhiều thuật toán nghiên cứu theo hướng sao chép phần lớn đường đi từ đường đi tốt cũ, từ đó giảm số thành phần lời giải cần phải xây dựng. Gambardella et al. đề xuất Enhanced ACS [26], trong đó để giảm thiểu thời gian chọn đỉnh tiếp theo, kiến sẽ có tỉ lệ đi thẳng theo đỉnh thuộc đường đi tốt nhất hiện tại, từ đó giúp EACS vượt qua ACS trong hầu hết mọi bộ dữ liệu được thực nghiệm. Skinderowicz et al. đề xuất FocusedACO [4], [5], thuật toán đặt ra số lần kiến chọn đỉnh theo hàm thông thường, và khi đã đủ lần chọn thì kiến sẽ đi theo đường đi cũ (chọn giữa đường đi tốt nhất vòng lặp trước và đường đi tốt nhất đến hiện tại) khi có thể, từ đó giảm thiểu đi số lần phải gọi hàm tính đỉnh tiếp theo. Hai nghiên cứu trên dựa trên quan sát thực tế từ thực nghiệm: hầu hết đường đi kiến chọn ở giai đoạn sau của thuật toán bị trùng lặp với đường đi tốt nhất, nên sao chép trực tiếp hầu hết cạnh từ đường đi tốt và chỉ điều chỉnh một ít là hợp lý. Không những vậy, tối ưu này cũng hạn chế

số lượng thay đổi được tạo ra bởi kiến, qua đó giảm thiểu những “cú nhảy” quá xa trong không gian tìm kiếm, thay vào đó khai thác được lời giải hiện tại tốt hơn.

3.4. Tìm kiếm cục bộ với mọi lời giải

Tìm kiếm cục bộ dựa trên tính chất của bài toán, đặc biệt là 2-opt, đã được áp dụng kết hợp với ACO trong các thuật toán kinh điển ban đầu. Tuy nhiên, với độ phức tạp cao, 2-opt chỉ được áp dụng kết hợp với ACO ở một số vòng lặp, nhằm giảm thiểu số lần tìm kiếm cục bộ. Để có thể thực hiện tìm kiếm cục bộ thường xuyên hơn, cần điều chỉnh 2-opt để nó trở nên đơn giản hơn nhưng nhanh hơn. FocusedACO (Skinderowicz et al.) đề xuất lưu lại danh sách các cạnh bị thay đổi, chỉ những cạnh bị thay đổi so với đường đi trước đó (đã trải qua tìm kiếm cục bộ) mới được áp dụng 2-opt, từ đó giảm độ phức tạp của tìm kiếm cục bộ. Một tối ưu khác là trong quá trình chọn cạnh để xét thay đổi, thay vì duyệt qua n đỉnh, 2-opt có thể sử dụng danh sách ứng viên đã được áp dụng trước đó, chỉ duyệt qua các cạnh thuộc danh sách ứng viên để thử thay đổi. Với số lượng ứng viên là một hằng số và kiểm soát số cạnh bị thay đổi nhỏ, tìm kiếm cục bộ 2-opt có thể áp dụng cho mọi lời giải được tạo ra trong thuật toán, thay vì chỉ áp dụng trong một số vòng lặp như trước đây.

4. Đề xuất thuật toán mới

Dựa trên những tối ưu và nhận xét ở trên, kết hợp với một số quan sát và điều chỉnh mới, phần này sẽ đề xuất một thuật toán mới nhằm giải quyết các bộ dữ liệu lớn hơn 100,000 đỉnh, đồng thời vẫn đủ linh hoạt để xử lý các bộ dữ liệu nhỏ hơn. Dưới đây là một số kĩ thuật tối ưu mới:

4.1.1. Ép buộc sự thay đổi của đường đi

4.1.1.1. Nhận xét

Dựa trên cài đặt của quá trình thay đổi lời giải ở trên, ta có thể nhận xét như sau: thời gian chạy của quá trình điều chỉnh đường đi phụ thuộc trực tiếp vào điều kiện khi từ u chọn được v , cạnh (u, v) có tồn tại trong đường đi gốc hay không. Chỉ khi không tồn tại cạnh, thuật toán mới thừa nhận đây là một thay đổi. Do đó, thời gian chạy của thuật toán tăng khi hầu hết các đỉnh được chọn là những đỉnh trong đường đi ban đầu. Vì vậy, thuật toán đề xuất thêm hai tối ưu nhằm thay đổi hành vi

của kiến, từ đó giảm thời gian chạy mà đánh đổi ít chất lượng lời giải nhất có thể. Điều này cho phép tăng số vòng lặp và kiến để cải thiện chất lượng lời giải.

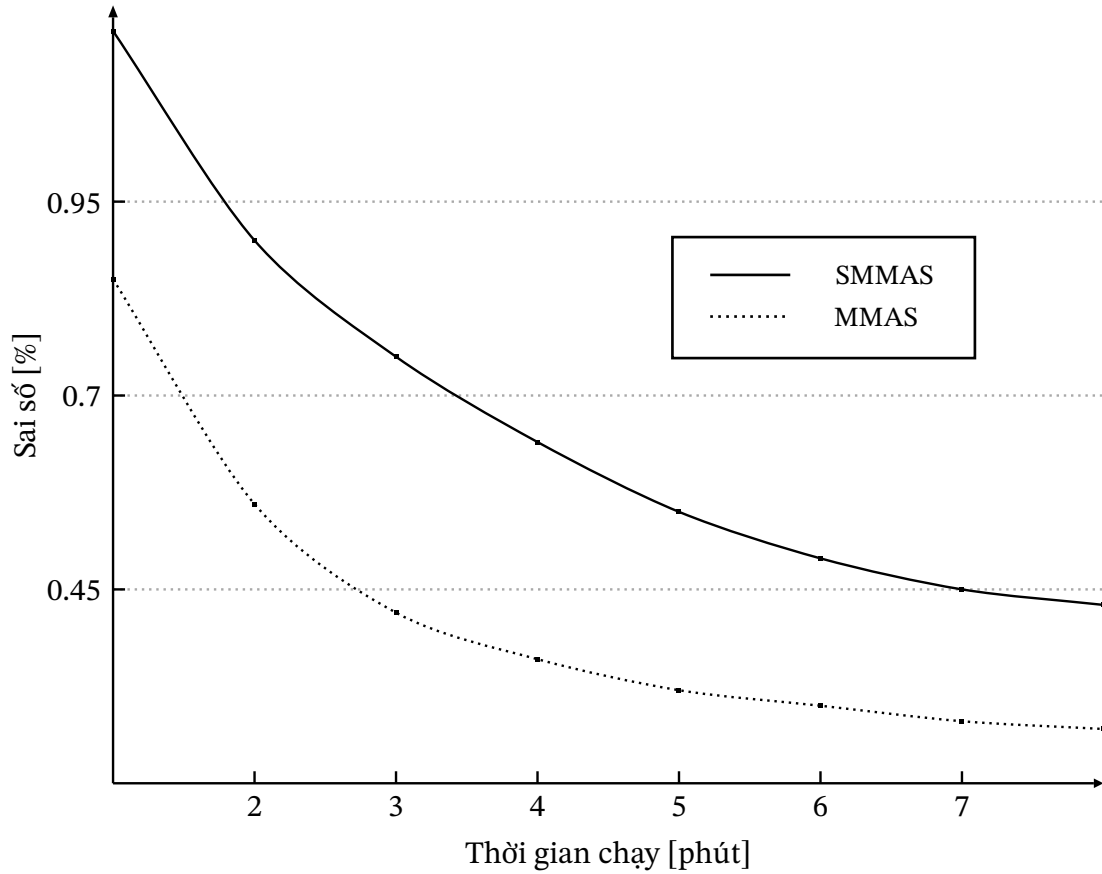
4.1.1.2. Cập nhật Max-Min Tròn

Một cách để thay đổi hành vi của kiến là tác động trực tiếp đến ma trận mùi, hay nói cách khác, điều khiển độ chênh lệch của các vết mùi mà ảnh hưởng ít tới chất lượng lời giải. Thuật toán đề xuất sử dụng cập nhật Max-Min tròn (SMMAS [6]) thay vì cập nhật MMAS truyền thống. Trong thuật toán SMMAS, vết mùi được cập nhật theo cách sau:

$$\tau_{i,j} := \begin{cases} \tau_{i,j} * (1 - p) + p * \tau_{\max} & \text{nếu } (i, j) \in w(t) \\ \tau_{i,j} * (1 - p) + p * \tau_{\min} & \text{nếu } (i, j) \notin w(t) \end{cases}$$

Giá trị vết mùi được tăng thêm không phụ thuộc vào độ dài đường đi nữa, thay vào đó vết mùi được tăng (hoặc giảm) tròn mìn về ($\tau_{\min}$ và $\tau_{\max}$).

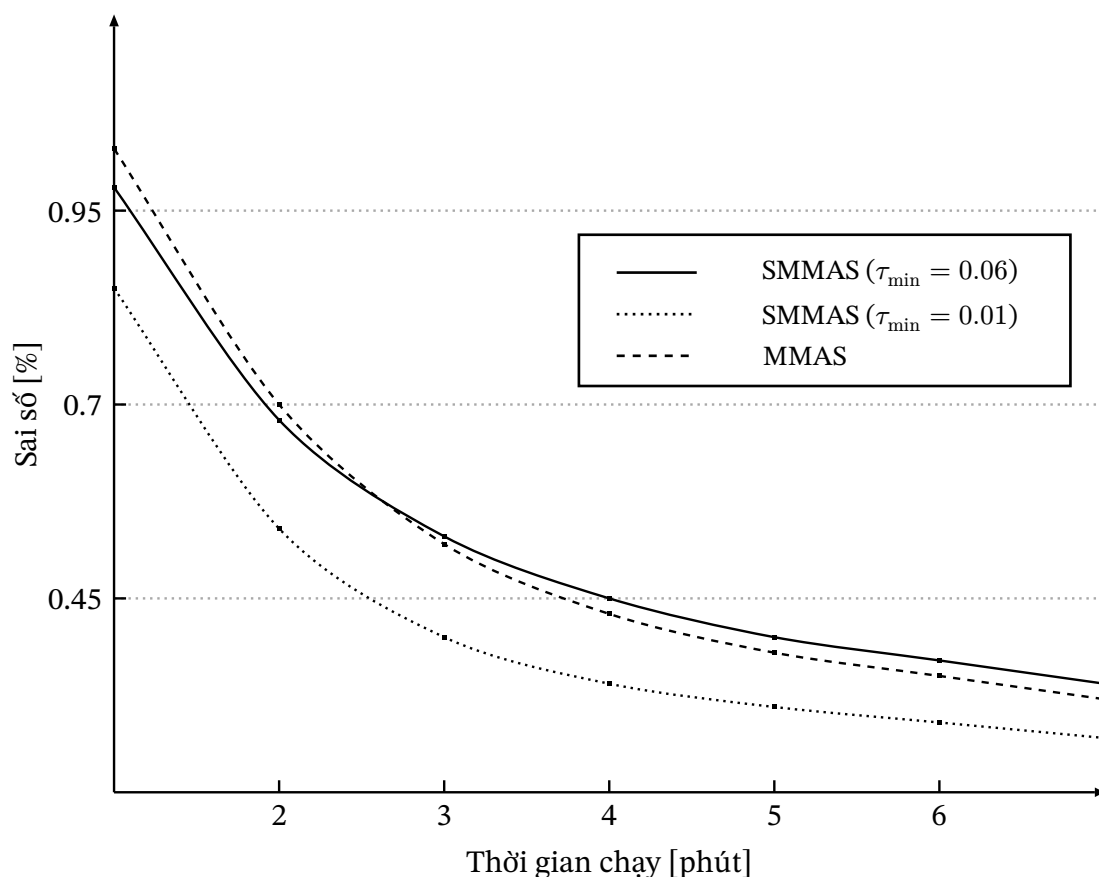
Ngoài tác động và cải thiện chất lượng lời giải, thay đổi cách cập nhật vết mùi từ MMAS sang SMMAS cũng cho thuật toán kiểm soát ma trận mùi tốt hơn bằng cách điều chỉnh giá trị $\tau_{\min}$. Thử nghiệm cho thấy giá trị $\tau_{\min}$ thấp hơn giúp tăng chất lượng lời giải, nhưng đồng thời làm tăng thời gian chạy. Điều này có thể giải thích như sau: $\tau_{\min}$ thấp đồng nghĩa với danh sách mùi sẽ bị tập trung vào những đỉnh có mật độ vết mùi cao nhất, khiến cho kiến gần như luôn luôn chọn những đỉnh đó và làm cho thời gian kiến hoàn thành quá trình điều chỉnh lời giải lâu hơn, tuy nhiên lại khai thác được lời giải tốt hơn. Ngược lại, khi $\tau_{\min}$ cao hơn, kiến có nhiều khả năng chọn những đỉnh ít vết mùi hơn, dẫn đến thay đổi lời giải nhanh hơn, thời gian chạy nhỏ hơn. Tạm thời, thuật toán đề xuất để $\tau_{\max} = 1, \tau_{\min} = n^{0.25}$.



So sánh giữa cách cập nhật mới với MMAS, thực nghiệm trên bộ test mona-lisa100K

Thay đổi này cải tiến khả năng kiểm soát hành vi của kiến, giúp xây dựng lời giải nhanh hơn, qua đó thời gian chạy thấp hơn với cùng số lượng vòng lặp với MMAS.

Giá trị của $\tau_{\min}$ là một tham số quan trọng trong cân bằng hai tính chất *khám phá* và *khai phá* của thuật toán. Kết quả thử nghiệm trên bộ dữ liệu *pla85900* thuộc TSPLIB cho thấy giá trị $\tau_{\min} = n^{0.25} \approx 0.06$ đưa ra kết quả trung bình kém hơn MMAS thông thường. Tuy nhiên, khi đặt $\tau_{\min} = 0.01$, thuật toán tiếp tục đưa ra chất lượng lời giải tốt hơn so với MMAS.

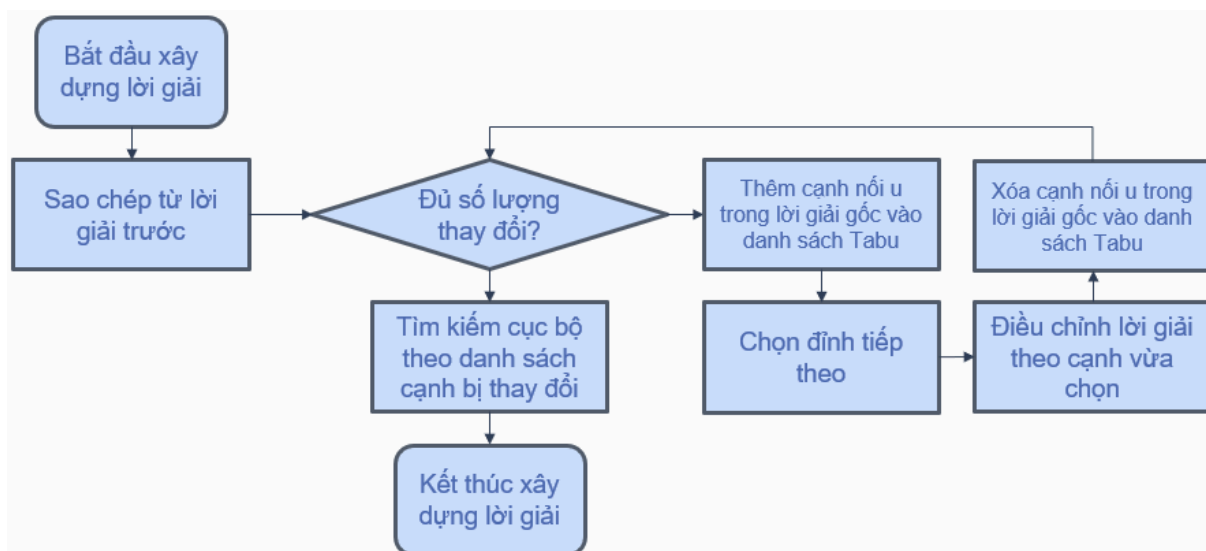


So sánh giữa MMAS và SMMAS với tác động của $\tau_{\min}$ trên bộ dữ liệu pla85900

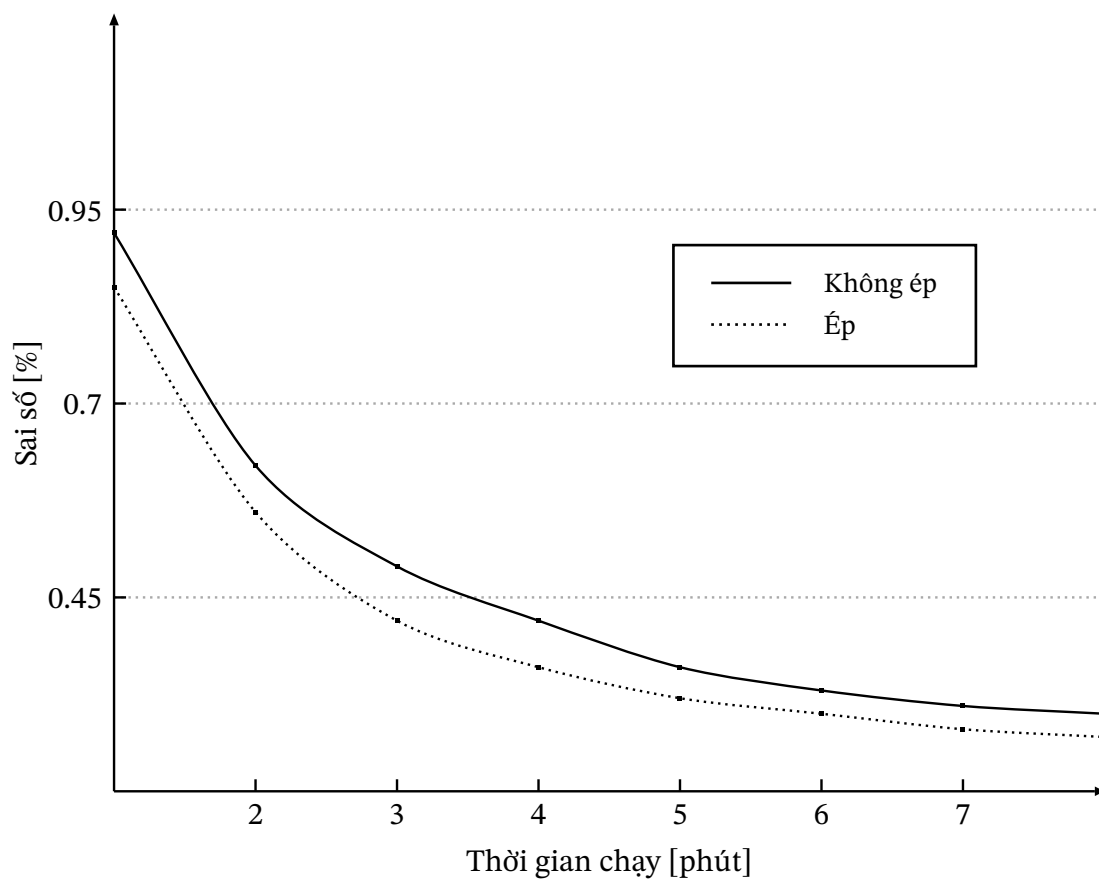
Điều này có thể giải thích được bằng nhu cầu cân bằng 2 yếu tố *khám phá* và *khai thác* trong ACO có thể thay đổi tùy thuộc vào không chỉ kích thước của bộ dữ liệu, mà cả tính chất của bộ dữ liệu đó. Tham số $\tau_{\min}$ được thiết đặt bằng $n^{0.25}$, nhằm tổng quát kết quả thực nghiệm trên các bộ dữ liệu khác nhau, mặc dù có thể chưa phải là giá trị tốt cho tất cả bộ dữ liệu.

4.1.1.3. Ép thay đổi sớm

Một tiếp cận khác nhằm giảm thời gian xây dựng lời giải của thuật toán kiến là ép kiến điều chỉnh lời giải, bằng cách buộc kiến chọn đỉnh tiếp theo trong những đỉnh ngoại trừ đỉnh hiện tại trong đường đi. Nhờ vậy, lượt chọn đầu tiên của đường đi kiến sẽ chọn một đỉnh khác với đường hiện tại và khả năng kiến thay đổi lời giải so với lời giải cũ. Qua thực nghiệm trên các bộ dữ liệu lớn, điều này đã được chứng minh là một thay đổi hợp lý giúp giảm thời gian chạy thuật toán mà gần như không ảnh hưởng đến chất lượng lời giải.



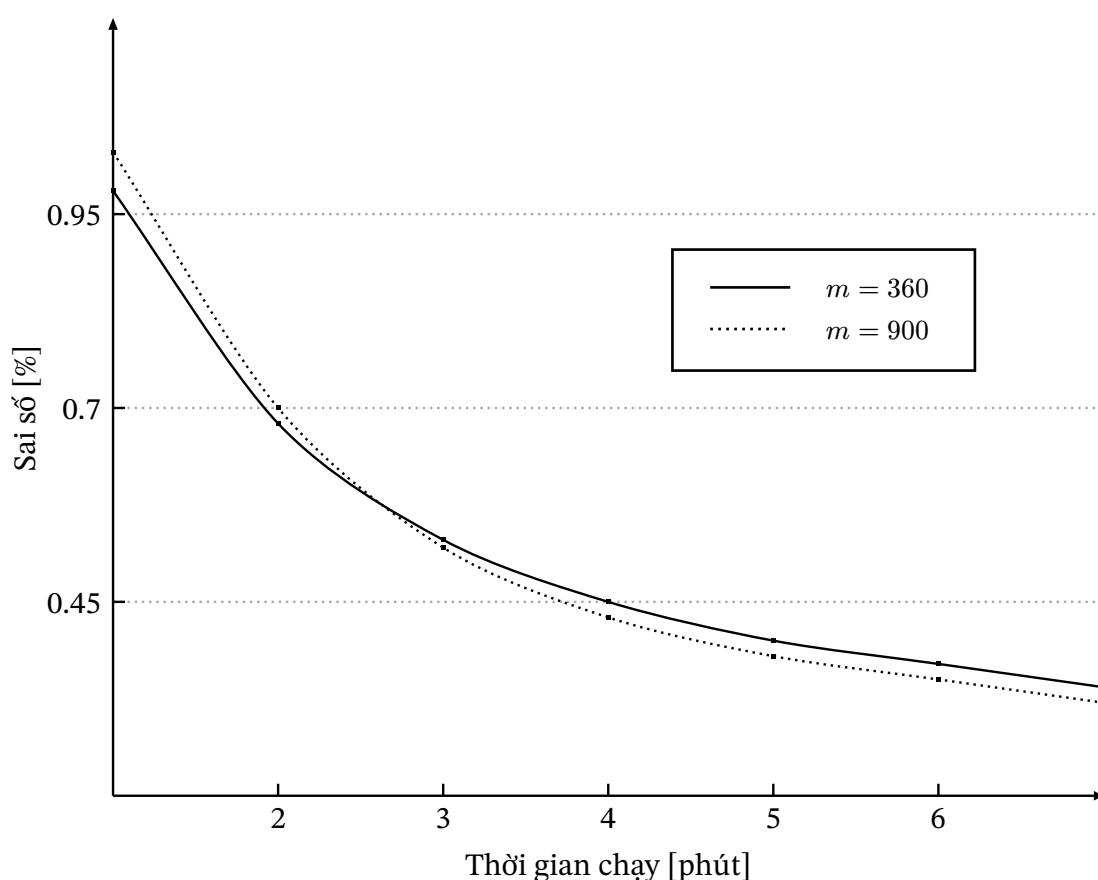
Ép kiến thay đổi lời giải ngay từ đầu



So sánh giữa không ép và ép thay đổi, thực nghiệm trên bộ test mona-lisa100K

4.1.2. Cập nhật số lượng kiến động

Trong ACO, số lượng kiến là một tham số rất quan trọng và có vai trò quyết định đến chất lượng của lời giải thuật toán. Khi có nhiều kiến, số lời giải được sinh ra ở mỗi vòng lặp cũng nhiều hơn, từ đó tăng tính khám phá của thuật toán và cải thiện chất lượng của lời giải. Tuy nhiên, nhu cầu về số kiến khác nhau trong ACO thay đổi tùy thuộc vào giai đoạn hiện tại của thuật toán. Cụ thể, ở giai đoạn đầu của thuật toán, chỉ cần ít kiến để tìm được đường lời giải đủ tốt. Ngược lại, khi tiến vào giai đoạn sau, nhu cầu về kiến tăng lên nhằm tăng khả năng khám phá và tránh sự tắc nghẽn của lời giải.

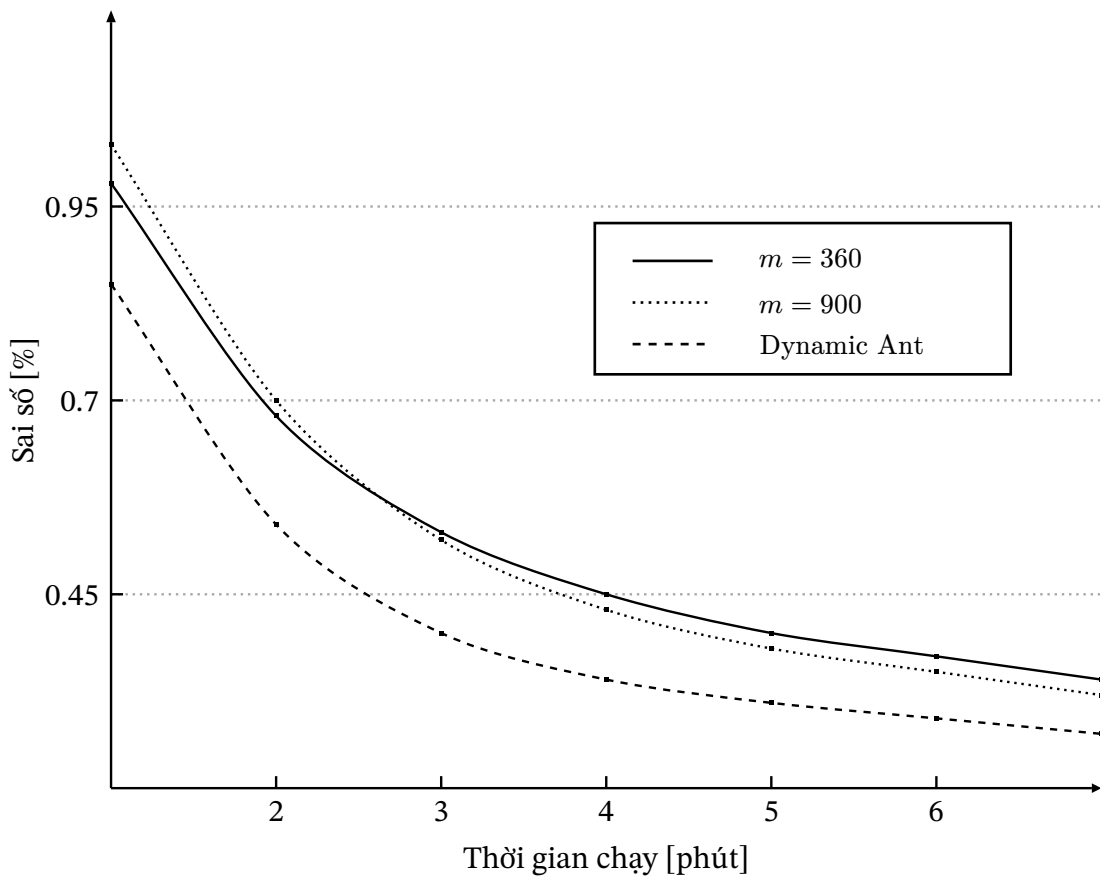


So sánh giữa 360 kiến và 900 kiến, trên bộ test mona-lisa100K

Có thể thấy, ở giai đoạn đầu, thuật toán với 360 kiến có hiệu quả theo thời gian tốt hơn so với 900 kiến. Tuy nhiên, ở giai đoạn sau, lượng kiến nhỏ như vậy không thể theo kịp nhu cầu của thuật toán, khi lời giải đã rơi vào tình trạng tắc nghẽn và

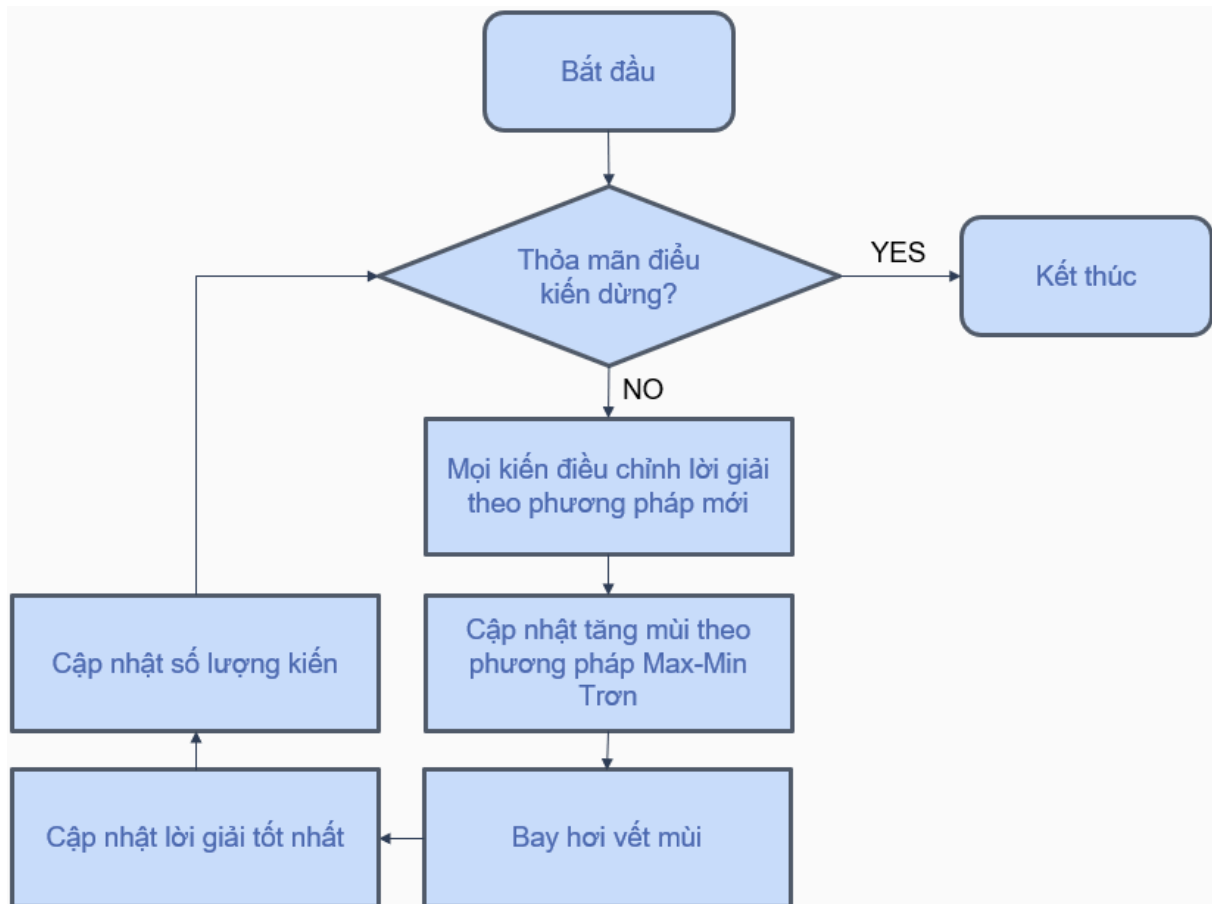
cần nhiều kiến hơn để khám phá lời giải tốt hơn. Giai đoạn sau đó, thuật toán 900 kiến đã vượt qua thuật toán 360 kiến.

Nghiên cứu đề xuất một cải tiến như sau: cập nhật và thay đổi số kiến theo tiến trình chạy của thuật toán. Ở giai đoạn đầu khi lời giải còn ít tắc nghẽn, có thể sử dụng ít kiến để nhanh chóng khám phá lời giải. Trong giai đoạn sau, số lượng kiến được tăng lên để phù hợp với nhu cầu khám phá. Theo đó, thuật toán bổ sung thêm tham số steps, sau mỗi steps vòng lặp thì số kiến sẽ được nhân đôi. Dưới đây là biểu đồ so sánh trước và sau khi thêm cải tiến, với giải thuật kiến động có số kiến khởi điểm là $m = 432$:



So sánh giữa 360 kiến và 900 kiến với kiến động, trên bộ test mona-lisa100K

Thuật toán kiến động rõ ràng vượt trội hơn so với kiến tĩnh khi số kiến được điều chỉnh phù hợp với nhu cầu trong quá trình chạy của thuật toán.



Lược đồ thuật toán đề xuất

5. Thực Nghiệm

Động lực chính cho những tối ưu và thay đổi được đề xuất là áp dụng thuật toán Tối ưu đàn kiến (ACO) để giải quyết và tìm lời giải tốt cho các bộ dữ liệu TSP với quy mô lớn, lên đến hàng trăm nghìn đỉnh, điều mà chỉ một số ít công trình nghiên cứu về ACO-TSP đã khám phá. Các bộ dữ liệu được thực nghiệm bao gồm các bộ dữ liệu từ TSPLIB [27], TSP Test Data [28] và Art TSP [7]. Chúng đã được thực nghiệm và so sánh với kết quả tốt nhất hiện có. Đối với các bộ dữ liệu thuộc Art TSP [7] với số đỉnh từ 100,000 đến 200,000, kết quả thực nghiệm đã được so sánh với tất cả các thuật toán ACO hiện tại cho TSP lớn.

Để kiểm chứng hiệu quả của những thay đổi được đề xuất, thuật toán mới (được đặt tên là AACO - Alternation-Based Ant Colony Optimization) đã được thực nghiệm và chạy trên các bộ dữ liệu TSP lớn. Chương trình được chạy trên cấu hình máy tính với CPU 16 nhân Intel(R) Xeon(R) CPU E5-2697 v4 @ 2.30GHz và 64GB RAM.

Các bộ dữ liệu được chia thành hai nhóm: nhóm kích thước lớn (từ 30,000 đến 100,000 đỉnh) và nhóm kích thước rất lớn (trên 100,000 đỉnh). Các tham số của thuật toán được thiết lập như sau: $\alpha = 1$, $\beta = 1$, $\rho = 0.5$, $\tau_{\min} = n^{0.25}$, $\text{candidate_size} = 16$, $\text{backup_size} = 64$, $\text{ants} = 432$, $\text{steps} = 4000$ đối với bộ dữ liệu rất lớn và $\text{ants} = 1800$, $\text{iterations} = 12000$, $\text{target_changes} = 16$ đối với bộ dữ liệu lớn. Tuy nhiên, như đã nhận xét, hiện tại, thuật toán đặt $\tau_{\min} = n^{0.25}$ có thể không phải là tham số tốt cho tất cả các bộ dữ liệu được thực nghiệm. Tùy thuộc vào tính chất của từng bộ dữ liệu, thuật toán sẽ yêu cầu tính khám phá hoặc khai thác khác nhau. Do đó, bảng thử nghiệm dưới đây có thể không đưa ra kết quả tốt nhất có thể của thuật toán đối với nhiều bộ dữ liệu.

Trong các bảng thực nghiệm tiếp theo, các bộ dữ liệu được chạy và đánh giá qua 10 lần chạy, “%” thể hiện sai số của kết quả so với lời giải tốt nhất hiện có của mỗi bộ dữ liệu. Sai số được tính bằng công thức sau:

$$\text{Sai số} = \frac{\text{cost} - \text{best known}}{100} \%$$

5.1. Bộ dữ liệu với kích thước lớn (từ 30,000 đến 100,000 đỉnh)

Dưới đây là bảng kết quả thực nghiệm của thuật toán được đề xuất với những bộ dữ liệu có kích thước từ 30,000 100,000 đỉnh, được lấy từ tất cả những bộ dữ liệu có kích thước lớn thuộc TSPLIB [27] và TSP Test Data [28]:

Instances	Best tour	AACO			
		min cost	mean cost	max cost	avg time
Tnm30001	566,973,186	566,973,183	566,973,185	566,973,189	199
	[4]	(-2)	(-1)	(0.00%)	
pla33810	125,183	66500498	66576633	66652768	296
		(0.68)	(0.79)	(0.91)	
Tnm40000	756,254,121	756,254,119	756,254,121	756,254,124	401
	[4]	(-0.0%)	(0%)	(0.0%)	
rbz43748	125,183	126,000	126,083	126,157	448
		(0.65%)	(0.71%)	(0.77%)	
fht47608	125,104	126,000	125,952	125,997	452
		(0.65%)	(0.67%)	(0.71%)	

Tnm50002	945,535,600 [4]	945,535,598 (-2)	945,535,600 (0)	945,535,600 (3)	516
fna52057	147,789	148,850 (0.71%)	148,939 (0.77%)	149,020 (0.83%)	452
bna56769	158,078	159,208 (0.71%)	159,290 (0.76%)	159,409 (0.84%)	551
dan59296	158,082	16,6484 (0.67%)	166,596 (0.74%)	166,685 (0.79%)	576
Tnm60001	1134816445 [4]	1,134,816,445 (0)	1,134,816,447 (0.0%)	1,134,816,449 (0.0%)	401
pla85900	142,382,641	143,482,461 (0.77%)	143,560,264 (0.82%)	143,612,046 (0.86%)	423
Tnm100000	1,891,945,653 [4]	1891945649 (-4)	1891945651 (-2)	1891945653 (0)	832 (0.23h)

Kết quả thực nghiệm cho thấy, thuật toán AACO đạt được kết quả tốt trên tất cả các bộ dữ liệu được thử nghiệm, với sai số tối đa dưới 1% và sai số trung bình dưới 0.8%. Đặc biệt, trên bộ dữ liệu Tnm, thuật toán đã tìm được lời giải tốt hơn cả kết quả từ LKH [29], một thuật toán heuristic tốt nhất hiện nay, và kết quả từ FocusedACO [4]. Bộ dữ liệu Tnm đã được thiết kế và xây dựng nhằm giảm hiệu quả của các thuật toán heuristic dựa vào tính chất toán học như LKH hay Concorde, trong đó các đỉnh được sắp xếp trên một tứ diện [30]. Khi giải bộ dữ liệu Tnm với 200 đỉnh, Concorde chạy chậm hơn 1,000,000 lần so với bộ dữ liệu thông thường, đồng thời, LKH cũng gặp khó khăn khi giải những bộ dữ liệu này. Bằng cách tiếp cận bài toán theo một hướng tổng quát hơn, sử dụng ACO, thuật toán AACO đã đạt được chất lượng lời giải tốt hơn LKH [4]. Đường đi tốt nhất của các bộ dữ liệu đã được lấy từ FACO [4], và AACO cũng đã tìm ra những lời giải tốt hơn so với lời giải tốt nhất.

Trên các bộ dữ liệu khác, mặc dù không đạt được kết quả tốt như Tnm, nhưng kết quả vẫn khá khả quan. Mặc dù chỉ có số vòng lặp và kiến thức là khác nhau giữa các bộ dữ liệu này và bộ dữ liệu có kích thước rất lớn, thực nghiệm đã cho thấy kết quả đạt được là tốt: mọi bộ dữ liệu đều có sai số trung bình dưới 0.82%. Mặc dù tham số này được thiết kế để cải thiện lời giải cho các bộ dữ liệu lớn hơn 100,000 đỉnh, kết

quả đã chứng minh được hiệu quả và tính linh hoạt của thuật toán trên các bộ dữ liệu nhỏ hơn.

5.2. Bộ dữ liệu với kích thước rất lớn (trên 100,000 đỉnh)

Dưới đây là bảng kết quả thử nghiệm của thuật toán được đề xuất trên những bộ dữ liệu có kích thước trên 200,000 đỉnh, được thực nghiệm trên bộ dữ liệu Art TSP: [7]:

Instances	Best tour	AACO			
		min cost	mean cost	max cost	avg time
mona-lisa100K	5,757,191	5,769,575 (0.21%)	5,769,900 (0.22%)	5,770,087 (0.22%)	802 (0.22h)
van-gogh120K	6,543,609	6,558,668 (0.23%)	6,558,869 (0.23%)	6,559,021 (0.23%)	1030 (0.28h)
venus140K	6,810,665	6,826,743 (0.23%)	6,826,911 (0.23%)	6,827,089 (0.24%)	1259 (0.34h)
pareja160K	7,619,953	7,639,301 (0.25%)	7,639,517 (0.25%)	7,639,905 (0.26%)	1584 (0.44h)
courbet180K	7,888,731	7,910,685 (0.27%)	7,911,253 (0.28%)	7,911,689 (0.29%)	2075 (0.57h)
ear-ring200K	8,171,677	8,196,911 (0.30%)	8,197,462 (0.31%)	8,198,014 (0.32%)	2738 (0.76h)

Những bộ dữ liệu lớn hơn 100,000 đỉnh, đặc biệt là các bộ dữ liệu thuộc Art TSP với kích thước lên đến 200,000 đỉnh, đã đạt được kết quả rất tốt: sai số nằm trong khoảng từ 0.2% đến 0.3%, với thời gian chạy tối đa chỉ 45 phút. Theo hiểu biết của tác giả, đây là kết quả tốt nhất đã đạt được cho bộ dữ liệu này bằng các thuật toán ACO. Dưới đây là bảng so sánh AACO với một số thuật toán ACO khác đã được thử nghiệm trên những bộ dữ liệu lớn này, cụ thể là ACO-RPMM (Peake et al., 2019 [9]), PartialACO (Chitty et al., 2018 [10]), FocusedACO (Skinderowicz et al., 2023 [5]):

Instances	Best	ACO-	Par-		
	known	RPMM [9]	tialACO	FACO [5]	AACO
	tour [7]				
Tour length (err%) - time [h]					

mona-		–	•	5,781,105	5,769,900
lisa100K	5,757,191	(1.7%)	(5.5%)	(0.42%)	(0.22%)
		1.4h	1.1h	0.19h	0.22h
van-		–	–	6,574,545	6,558,668
gogh120K	6,543,609	(1.8%)	(5.8%)	(0.47%)	(0.23%)
		1.9h	1.5h	0.20h	0.28h
venus140K	6,810,665	(1.8%)	(5.8%)	(0.50%)	(0.23%)
		2.6h	2.1h	0.24h	0.34h
pareja160K	7,619,953	(1.9%)	–	(0.56%)	(0.25%)
		3.5h	–	0.32h	0.44h
courbet180K	7,888,731	(1.9%)	–	(0.63%)	(0.28%)
		4.5h	–	0.40h	0.57h
ear-		–	–	8,229,457	8,197,462
ring200K	8,171,677	(2.0%)	(7.2%)	(0.71%)	(0.31%)
		6.0h	5.1h	0.43h	0.76h

Thông qua so sánh, với kết quả trên 6 bộ dữ liệu Art TSP có từ 100,000 đến 200,000 đỉnh, thuật toán mới đạt sai số tốt nhất trong số tất cả các thuật toán ACO đã được thử nghiệm trên các bộ dữ liệu có kích thước như vậy. Tuy nhiên, so sánh vẫn bị hạn chế do hầu hết các thuật toán ACO được nghiên cứu trước đây không được thử nghiệm trên các bộ dữ liệu lớn hàng trăm nghìn đỉnh, thay vào đó, chúng thường chỉ được thử nghiệm trên các bộ dữ liệu có đến vài trăm đỉnh, tối đa là vài nghìn đỉnh [20]. Mặc dù vậy, những thuật toán này, mặc dù số lượng ít ỏi, đều đại diện cho các hướng tiềm năng để áp dụng ACO vào các bộ dữ liệu rất lớn, và có thể cạnh tranh hoặc thậm chí vượt qua các thuật toán hiện đại nhất hiện nay cho TSP như LKH hay GA-EAX. Trong số đó, thuật toán AACO được đề xuất trong nghiên cứu này đã đạt hiệu năng và chất lượng lời giải tốt nhất trong số tất cả các thuật toán ACO đã được thử nghiệm.



Figure 1: Một lời giải cho bộ dữ liệu mona-lisa100K được xây dựng bởi AACO

6. Kết luận

6.1. Nhận xét

Trong nghiên cứu này, tác giả đã nêu lên bài toán người chào hàng có kích thước rất lớn, một bài toán NP-Khó kinh điển, cũng như những thuật toán tốt nhất hiện nay để giải TSP với kích thước rất lớn. Nhận thấy tiềm năng của giải thuật tối ưu đàn kiến, nghiên cứu đã trình bày thuật toán gốc và giới thiệu những tối ưu gần đây của ACO so với giải thuật của Dorigo từ năm 1996. Qua đó, nghiên cứu đã quan sát, nhận xét và thử nghiệm những tối ưu cũ, đồng thời trình bày một số thay đổi và cải tiến mới, từ đó đề xuất thuật toán mới - Alternation-based Ant Colony Optimization (AACO). Bằng cách kết hợp lựa chọn các tối ưu hoàn thiện được nghiên cứu bởi các công trình trước đó và các tối ưu và điều chỉnh mới, thuật toán được đề xuất đã đạt được những thành tựu đáng kể, tìm được lời giải tốt nhất mới cho một số bộ dữ liệu và đạt được sai số tốt nhất trong tất cả những thuật toán ACO đã được thử nghiệm cho bộ dữ liệu Art TSP có từ 100,000 đến 200,000 đỉnh.

Giải thuật tối ưu đàn kiến là một phương pháp metaheuristic tiềm năng để giải quyết bài toán người chào hàng. Mặc dù cho đến thời điểm hiện tại, so với giải thuật di truyền tốt nhất (GA-EAX), ACO vẫn chưa thể tìm được những lời giải tốt nhất cho các bộ dữ liệu lớn với hàng trăm nghìn đỉnh, nhưng các nghiên cứu gần đây đã đẩy mạnh tiến bộ của ACO trong giải quyết quy mô này. AACO là một thuật toán

mới được đề xuất trong nghiên cứu này và với các đề xuất và điều chỉnh mà nó đem lại, nó là một bổ sung có giá trị cho nhóm các thuật toán ACO, đặc biệt là để giải quyết các bộ dữ liệu lớn.

Qua thực nghiệm, mặc dù không điều chỉnh thông số cho các bộ dữ liệu, chỉ phân chia thành hai nhóm lớn và rất lớn, thuật toán vẫn có thể đạt được kết quả tốt trên tất cả các bộ dữ liệu và đạt được sai số chỉ từ 0.2% đến 0.8%. Đáng chú ý hơn, các bộ dữ liệu có sai số cao là các bộ dữ liệu có kích thước dưới 100,000 đỉnh, như đã được nhận xét, chưa được điều chỉnh tham số phù hợp. Trên các bộ dữ liệu mà các tham số được đặt tốt, như 6 bộ dữ liệu Art TSP, cũng là 6 bộ dữ liệu có kích thước lớn nhất trong các bộ dữ liệu được thực nghiệm, với số đỉnh lên đến 200,000, thuật toán đã đạt được kết quả vượt trội so với tất cả các nghiên cứu và thực nghiệm của ACO từ trước đến nay.. [8], [10], [21], [9], [4], [5].

6.2. Hướng nghiên cứu trong tương lai

Giải thuật Alternation-based Ant Colony System (AACO) được đề xuất đã đạt được kết quả tốt cho nhiều bộ dữ liệu TSP lớn. Tuy nhiên, vẫn còn nhiều hướng nghiên cứu tiềm năng mở rộng từ nghiên cứu trên. Giá trị của $\tau_{\min}$, đại diện cho cận dưới của vết mùi, đã được chứng minh là đem đến ảnh hưởng rất lớn đến hiệu suất thuật toán và có thể được nghiên cứu và thử nghiệm nhằm hoàn thiện và tối ưu. Đồng thời, tương tự như tối ưu mới về số kiến động trong nghiên cứu trên, những tham số khác cũng có thể được cài đặt và thử nghiệm để thích nghi với nhu cầu của thuật toán trong những thời điểm khác nhau, đặc biệt là giá trị của $\tau_{\min}$ và *target_changes*, bởi tại các thời điểm khác nhau, nhu cầu tỉ lệ *khám phá* và *khai phá* là khác nhau. Một hướng nghiên cứu tiềm năng khác là sự kết hợp giữa tiếp cận của thuật toán dựa trên quá trình điều chỉnh đường đi cũ của AACO với thuật toán dựa trên quần thể như PartialACO (Chitty et al. [10]) và P-ACO (Guntsch et al. [21]). Mặc dù kết quả của của những thuật toán này trong thử nghiệm kém hơn so với AACO ([10], [21]), nhưng duy trì một tập những đường đi tốt nhất thay vì ma trận mùi là một tiếp cận có thể đưa đến kết quả tốt, khi nghiên cứu chỉ ra rằng khi kết hợp ACO với tìm kiếm cục bộ (LS), khi để tham số bay hơi $p = 1$ vẫn khiến cho thuật toán tìm được kết quả tương đương so với $p = 0.5$ [4]. Để $p = 1$ đồng nghĩa với loại bỏ hoàn toàn ma trận mùi, thay vào đó kiến chỉ biết về cạnh nằm trong đường đi tốt nhất ở vòng lặp trước (iteration best) hoặc tốt nhất hiện tại (global best) và cạnh

không nằm trong đó, tương tự như chỉ duy trì ít quần thể trong tiếp cận thuật toán dựa trên quần thể.

Trong tương lai, thuật toán cũng có thể được mở rộng hơn nữa về độ linh hoạt trong quy mô, từ đó có thể được thử nghiệm trên những bộ dữ liệu lớn hơn, ví dụ như bộ World TSP với 1,904,711 đỉnh. Giải thuật ACO là một giải thuật có nhiều tiềm năng chưa được khai phá để giải những bộ dữ liệu lớn, và nghiên cứu này hy vọng sẽ đưa ra một hướng tiếp cận mạnh mẽ để tối ưu và điều chỉnh ACO, thậm chí vượt qua những giải thuật hiện đại nhất hiện nay cho TSP như LKH và GA-EAX.

6.2.1. Credit

Trần Thành Đạt: Quan sát, thiết kế thuật toán và thực nghiệm. Thu thập và phân tích dữ liệu. Viết bài báo.

6.2.2. Lời cảm ơn

Lời cảm ơn đến thầy Đỗ Đức Đông với sự hỗ trợ của thầy khi nghiên cứu, và lời cảm ơn đến Rafał Skinderowicz [4], [5] đã hỗ trợ cài đặt và so sánh kết quả thực nghiệm.

Bibliography

- [1] K. L. Hoffman, M. Padberg, G. Rinaldi, and others, “Traveling salesman problem,” *Encyclopedia of operations research and management science*, vol. 1, pp. 1573–1578, 2013.
- [2] S. Lin, “Computer solutions of the traveling salesman problem,” *Bell System Technical Journal*, vol. 44, no. 10, pp. 2245–2269, 1965.
- [3] R. Matai, S. P. Singh, and M. L. Mittal, “Traveling salesman problem: an overview of applications, formulations, and solution approaches,” *Traveling salesman problem, theory and applications*, vol. 1, no. 1, pp. 1–25, 2010.
- [4] R. Skinderowicz, “Improving Ant Colony Optimization efficiency for solving large TSP instances,” *Applied Soft Computing*, vol. 120, p. 108653–108654, May 2022, doi: 10.1016/j.asoc.2022.108653.
- [5] R. Skinderowicz, “Focused ACO with node relocation procedure for solving large TSP instances,” *Procedia Computer Science*, vol. 225, pp. 2992–3000, 2023, doi: <https://doi.org/10.1016/j.procs.2023.10.292>.

- [6] D. D. Dong,.
- [7] C. S. Kaplan and R. Bosch, “TSP art,” pp. 301–308, 2005.
- [8] P. A. Martínez and J. M. García, “ACOTSP-MF: A memory-friendly and highly scalable ACOTSP approach,” *Engineering Applications of Artificial Intelligence*, vol. 99, p. 104131–104132, 2021, doi: <https://doi.org/10.1016/j.engappai.2020.104131>.
- [9] J. Peake, M. Amos, P. Yiapanis, and H. Lloyd, “Scaling techniques for parallel ant colony optimization on large problem instances,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019, pp. 47–54.
- [10] D. M. Chitty, “Applying ACO to large scale TSP instances,” in *Advances in Computational Intelligence Systems: Contributions Presented at the 17th UK Workshop on Computational Intelligence, September 6-8, 2017, Cardiff, UK*, 2018, pp. 104–118.
- [11] J. E. Bell and P. R. McMullen, “Ant colony optimization techniques for the vehicle routing problem,” *Advanced engineering informatics*, vol. 18, no. 1, pp. 41–48, 2004.
- [12] G. Leguizamon and Z. Michalewicz, “A new version of ant system for subset problems,” in *Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406)*, 1999, pp. 1459–1464.
- [13] H. Nezamabadi-Pour, S. Saryazdi, and E. Rashedi, “Edge detection using ant algorithms,” *Soft computing*, vol. 10, pp. 623–628, 2006.
- [14] J. Gomez *et al.*, “Ant colony system algorithm for the planning of primary distribution circuits,” *IEEE Transactions on power systems*, vol. 19, no. 2, pp. 996–1004, 2004.
- [15] M. Dorigo and T. Stützle, *Ant colony optimization: overview and recent advances*. Springer, 2019.
- [16] M. Dorigo, V. Maniezzo, and A. Coloni, “Ant system: optimization by a colony of cooperating agents,” *IEEE transactions on systems, man, and cybernetics, part b (cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996.

- [17] M. Dorigo and L. M. Gambardella, “Ant colony system: a cooperative learning approach to the traveling salesman problem,” *IEEE Transactions on evolutionary computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [18] T. Stützle and H. H. Hoos, “MAX–MIN ant system,” *Future generation computer systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [19] S. Gupta, A. Rana, and V. Kansal, “Comparison of Heuristic techniques: A case of TSP,” in *2020 10th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, 2020, pp. 172–177.
- [20] H. Ismkhan, “Effective heuristics for ant colony optimization to handle large-scale problems,” *Swarm and Evolutionary Computation*, vol. 32, pp. 140–149, 2017.
- [21] M. Guntsch and M. Middendorf, “A population based approach for ACO,” in *Applications of Evolutionary Computing: EvoWorkshops 2002: EvoCOP, EvoIASP, EvoSTIM/EvoPLAN Kinsale, Ireland, April 3–4, 2002 Proceedings*, 2002, pp. 72–81.
- [22] M. López-Ibáñez and C. Blum, “Beam-ACO for the travelling salesman problem with time windows,” *Computers & operations research*, vol. 37, no. 9, pp. 1570–1583, 2010.
- [23] M. Scianna, “The AddACO: A bio-inspired modified version of the ant colony optimization algorithm to solve travel salesman problems,” *Mathematics and Computers in Simulation*, vol. 218, pp. 357–382, 2024.
- [24] W. Cook, “TSP art instances,” [Online]. Available: <https://www.math.uwaterloo.ca/tsp/data/art>
- [25] T. Stützle, M. Dorigo, and others, “ACO algorithms for the traveling salesman problem,” *Evolutionary algorithms in engineering and computer science*, vol. 4, pp. 163–183, 1999.
- [26] L. M. Gambardella, R. Montemanni, and D. Weyland, “Coupling ant colony systems with strong local searches,” *European Journal of Operational Research*, vol. 220, no. 3, pp. 831–843, 2012.
- [27] H. University, “TSP Test Data,” [Online]. Available: <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/>

- [28] U. of Waterloo, “TSP Test Data,” [Online]. Available: <https://www.math.uwaterloo.ca/tsp/data/>
- [29] K. Helsgaun, “Best LKH Solutions for Some Large Tnm Instances,” [Online]. Available: <http://webhotel4.ruc.dk/~keld/research/LKH/Best%20LKH%20solutions%20for%20Tnm%20instances.pdf>
- [30] S. Hougardy and X. Zhong, “Hard to solve instances of the Euclidean Traveling Salesman Problem,” *Mathematical Programming Computation*, vol. 13, no. 1, pp. 51–74, Mar. 2020, doi: 10.1007/s12532-020-00184-5.